

방 사진 기반 2D 평면도 자동 생성

목차 (Table of Contents)

1. 전체 EPIC
2. 스프린트 계획
 - 스프린트 1주차 (2025.07.08 ~ 07.12)
 - 일별 스크럼
 - 7월 8일 (화)
 - 7월 9일 (수)
 - 7월 10일 (목)
 - 7월 11일 (금)
 - 스프린트 2주차 (2025.07.14 ~ 07.18)
 - 일별 스크럼
 - 7월 14일 (월)
 - 7월 15일 (화)
 - 7월 16일 (수)
 - 7월 17일 (목)
 - 7월 18일 (금)
 - 스프린트 3주차 (2025.07.21 ~ 07.25)
 - 일별 스크럼
 - 7월 21일 (월)
 - 7월 22일 (화)
 - 7월 23일 (수)
 - 7월 24일 (목)
 - 7월 25일 (금)
 - 스프린트 4주차 (2025.07.28 ~ 08.01)
 - 일별 스크럼
 - 7월 28일 (월)
 - 7월 29일 (화)
 - 7월 30일 (수)
 - 7월 31일 (목)
 - 8월 1일 (금)
 - 스프린트 5주차 (2025.08.04 ~ 08.08)
 - 일별 스크럼
 - 8월 4일 (월)
 - 8월 5일 (화)

- 8월 6일 (수)
- 8월 7일 (목)
- 8월 8일 (금)
- 스프린트 6주차 (2025.08.11 ~ 08.15)
 - 일별 스크럼
 - 8월 11일 (월)
 - 8월 12일 (화)
 - 8월 13일 (수)
 - 8월 14일 (목)
 - 8월 15일 (금)
- 스프린트 7주차 (2025.08.18 ~ 08.22)
 - 일별 스크럼
 - 8월 18일 (월)
 - 8월 19일 (화)
 - 8월 20일 (수)
 - 8월 21일 (목)
 - 8월 22일 (금)

1. 전체 EPIC

사용자가 업로드한 방 사진을 기반으로 YOLOv11, MiDaS, RoomNet 등의 AI 모델을 활용해 가구를 감지하고, 깊이-레이아웃 정보를 추정하여 실제 방의 평면도(2D) 구조를 자동으로 시각화하는 웹 서비스를 개발한다. 주요 작업에는 모델 연동, 프론트 UI/UX 구성, API 구축, 도커/쿠버네티스 배포 및 CI/CD 자동화가 포함된다.

EPIC-1: AI 기반 분석 파이프라인 구성

EPIC-2: MiDaS 기반 실측 거리 추정 기능 구현

EPIC-3: 정확도 향상 및 안정화 작업

EPIC-4: 팀 통합 작업 및 웹애플리케이션 완성

EPIC-5: Docker 기반 배포 환경 구축 및 CI/CD 구성

EPIC-6: Kubernetes 도입 및 고급 배포 환경 구축

EPIC-7: 최종 발표 준비 및 프로젝트 완성

2. 스프린트 계획

2.1. 스프린트 1주차 (2025.07.08 ~ 07.12)

2.1.1. 1주차 EPIC : AI 기반 분석 파이프라인 구성

YOLOv11, MiDaS, RoomNet 등 AI 모델을 테스트하고, 업로드된 방 사진을 분석하는 초기 파이프라인을 구성한다.

FastAPI 서버 구조 설계, 사진 업로드 UI, 총고 입력 필드 구축까지 포함.

2.1.2. 일별 스크럼

2.1.2.1. 7월 8일 (화)

- **어제 한 일:** (N/A – 주간 시작일)
 - **오늘 할 일:**
 - AI: YOLOv11 사전 학습된 모델 로딩 및 샘플 이미지 추론 테스트
 - FastAPI: `/api/layout/detect` 기본 스텁레톤 구성
 - **프론트엔드: 사진 업로드 컴포넌트 및 총고 입력 필드 UI 구현 시작**
-

- **이슈:** -
- **블로커:** -

2.1.2.2. 7월 9일 (수)

- **어제 한 일:**
 - YOLOv11 모델 추론 테스트 성공
 - FastAPI 기본 구조 및 업로드 엔드포인트 생성
 - 프론트엔드 파일 업로드 드래그앤드롭 UI 구축 완료
- **오늘 할 일:**
 - FastAPI `/analyze-room` 에 입력 포인트 (ceiling_y, floor_y, left_x, right_x) 기반 room size 계산 적용
 - OpenCV 활용한 벽 높이 픽셀 계산 로직 추가
 - 좌우 벽 포인트 기반 가로 너비 계산 로직 적용
 - 프론트엔드에서 방 크기(cm 단위) 및 감지 결과 시각화 연동 검증
- **이슈:**
 - 총고 기반 환산 시 기준 객체(Bed 등)가 없을 경우 좌우 벽 포인트로 보완하는 방식으로 결정 중
- **블로커:**

- 이미지가 왜곡되어 찍혔을 경우 벽 높이 픽셀 길이 기준 환산의 정밀도 저하 우려 있음 → 추후 보정 알고리즘 필요

2.1.2.3. 7월 10일 (목)

• 어제 한 일:

- 층고값 기반 비율 환산 로직 기본 구현
- 프론트엔드 층고 입력 필드 + 유효성 검사 구현

• 오늘 할 일:

- FastAPI에서 `/estimate-room-size`` API 설계 시작
 - 사용자로부터 4개 지점(바닥, 천장, 좌/우 벽)의 Pixel + Depth 입력 받아 크기 추정 구조 정의
- MiDaS 기반 depth 추정값을 3D 좌표로 환산하는 함수 설계 (`to_3d_point`)

• 이슈:

- 프론트에서 수집된 좌표가 depth와 함께 정확히 전달되지 않음
- 클릭한 좌표에 따라 추정되는 크기가 편차 발생

• 블록어:

- MiDaS가 제공하는 깊이값이 정규화된 상대값이기 때문에 실제 단위 거리 환산 불가능
- 기준이 되는 실제 층고가 주어지지 않으면 스케일 보정이 어려움

2.1.2.4. 7월 11일 (금)

• 어제 한 일:

- ``/estimate-room-size`` API 1차 버전 완성
 - 입력된 4점(Point3D) → MiDaS 기반 3D 좌표로 변환
 - 3D 거리 계산 후, 실제 층고 230cm를 기준으로 스케일 보정
 - 가로·세로 크기(cm), 기존 2D 방식 대비 개선률 포함 응답 구성

• 오늘 할 일:

- ``/get-depth-at-point`` , ``/depth-distance`` 등 보조 API 연결 및 테스트
- `FastAPI 로그 개선 (좌표 정보, 변환 과정 로그 출력)
- `MiDaS depth 변환 공식을 튜닝하여 상대 오차 보정 시도

• 이슈:

- 동일한 이미지라도 클릭 위치에 따라 깊이값 편차 심함
- 정규화된 좌표 → 실좌표 변환시 focal length 가정이 맞지 않으면 왜곡 발생

- 블로커:

2.2. 스프린트 2주차 (2025.07.14 ~ 07.18)

2.2.1. 2주차 EPIC : MiDaS 기반 실측 거리 추정 기능 구현

MiDaS depth 결과를 이용해 사용자가 선택한 좌표들로부터 실제 방 크기를 추정하고, 기존 방식 대비 개선된 정확도를 확보한다.

스케일 보정, 3D 좌표 계산, 거리 비교 등을 포함한다.

2.2.2. 일별 스크럼

2.2.2.1. 7월 14일 (월)

- 어제 한 일:

- ``/estimate-room-size` API 고도화
 - depth 기반 3D 변환 + 3D 거리 추정 + 스케일 보정 흐름 정리
 - 기존 2D 방식과의 비교값 제공(improvement_percent)
 - depth range, 시점 왜곡 강도(perspective_strength) 판단 로직 추가

- 오늘 할 일:

- 프론트엔드와 /estimate-room-size 연결
 - 클릭한 좌표와 depth를 백엔드에 전달하여 실시간 방 크기 계산
- /depth-distance 와 로직 공통화 및 모듈 분리 작업
- Canvas에 2D 평면도 형태로 시각화 시작

- 이슈:

- 입력된 4점의 정렬이나 각도가 실제 방 구조와 다를 경우 결과가 왜곡됨
- depth range가 좁은 경우 실측 정확도가 떨어짐

- 블로커:

2.2.2.2. 7월 15일 (화)

- 어제 한 일:

- /estimate-room-size 에 3D 기반 좌표 계산 및 실측 보정 로직 적용 완료
 - to_3d_point() 로 MiDaS depth → 3D world coordinate 변환 구현
 - 기준층고(2.3m)를 기준으로 scale factor 계산 → width/depth 보정
- 2D vs 3D 계산 결과 비교 로그 출력 (개선율 계산 포함)
- 방 크기 추정 결과에 perspective_strength , depth_range , 스케일 팩터 등 정보 추가

- **오늘 할 일:**

- 정확도 향상을 위한 구조 개선
 - 다양한 계수 테스트 및 상대 오차 분석
- MiDaS가 출력하는 depth의 상대성 극복을 위한 벤치마크 테스트
 - 다양한 방 이미지에서 실제값 대비 예측값 비교
 - 평균 scale factor, 편차 분석 → 보정 근거 수립

- **이슈:**

- **블로커:**

2.2.2.3. 7월 16일 (수)

- **어제 한 일:**

- 정확도 향상을 위한 구조 개선
- MiDaS가 출력하는 depth의 상대성 극복을 위한 벤치마크 테스트

- **오늘 할 일:**

- 클릭 기반 거리 실험 및 로그 수집
 - /get-depth-at-point API 사용하여 다수의 클릭 지점 depth 수집
 - 실제 거리와 비교해 적절한 보정 수식 실험 ($a/(depth+b)$ 등)
 - 회귀 방식 2~3가지 실험 및 로그 저장
- 3D 거리 vs 2D 거리 비교
 - /depth-distance API로 보정 전후 거리 차이 확인
 - 같은 지점 쌍에 대해 2D 거리와 3D 거리 차이 시각화

- **이슈:**

- **블로커:**

2.2.2.4. 7월 17일 (목)

- **어제 한 일:**
 - 클릭 기반 거리 실험 및 로그 수집 완료
 - 2D/3D 거리 비교 데이터 확보
- **오늘 할 일:**
 - 수집된 로그 기반으로 최적의 보정 수식 도출
 - `/estimate-room-size` API에 최종 보정 로직 적용
 - 프론트엔드에서 보정 전/후 거리 비교 시각화 기능 구현
- **이슈:**
 - 특정 구도(예: 광각)에서 왜곡 보정 효과 미미
- **블로커:**

2.2.2.5. 7월 18일 (금)

- **어제 한 일:**
 - API에 최종 보정 로직 적용 완료
 - 프론트엔드 비교 시각화 기능 1차 구현
- **오늘 할 일:**
 - 다양한 테스트 이미지로 최종 정확도 검증
 - 2주차 스프린트 결과 정리 및 회고
 - 3주차 계획 구체화 (캔버스 시각화)
- **이슈:**
- **블로커:**

2.3. 스프린트 3주차 (2025.07.21 ~ 07.25)

2.3.1. 3주차 EPIC : 2D/3D 결과 시각화 및 UI, 정확도 향상 및 안정화 작업

방 사진을 기반으로 생성된 2D 평면도와 3D 모델을 명확하게 시각화하는 것입니다. 사용자가 쉽게 이해할 수 있는 직관적인 UI를 구축하고, 2D 및 3D 결과물의 정확도를 향상시키며, 서비스의 안정성을 확보하는 데 중점을 둡니다.

2.3.2. 일별 스크럼

2.3.2.1. 7월 21일 (월)

- **어제 한 일:** 2주차 스프린트 결과 정리 및 회고.
- **오늘 할 일:**
 - 현재 구현된 방 측정 및 시각화 기능(이미지 업로드, 깊이 맵 생성, 방 크기 추정, 2D/3D 시각화)에 대한 전반적인 검토.
 - jira 재설정
- **이슈:**
- **블로커:** -

2.3.2.2. 7월 22일 (화)

- **어제 한 일:** jira 재설정
- **오늘 할 일:**
 - 방 창문 사진 3D 뷰어 적용
 - 측정된 방 데이터를 활용한 2D 평면도 인터랙션 강화 및 3D 뷰어 시각적 개선 계획 수립.
 - 백엔드에서 평면도 데이터(방 외곽선, 가구 위치 등)를 JSON으로 반환하는 API 스텁레톤 구성 (인테리어 디자인 활용 목적).
- **이슈:**
- **블로커:** -

2.3.2.3. 7월 23일 (수)

- **어제 한 일:**
 - 방 창문 사진 3D 뷰어 적용 완료
 - 2D 평면도 인터랙션 강화 계획 수립
 - 백엔드 평면도 데이터 JSON 반환 API 스텁레톤 구성
- **오늘 할 일:**
 - 창문 감지 기능 개발 및 3D 뷰어 통합
 - 가구 배치 시스템 기본 구조 설계
 - 프론트엔드 UI/UX 개선 작업
- **이슈:**
 - 창문 감지 정확도 조정 필요
- **블로커:**

2.3.2.4. 7월 24일 (목)

- **어제 한 일:**
 - 창문 감지 기능 개발 및 3D 뷰어 통합 완료
 - 가구 배치 시스템 기본 구조 설계
 - 프론트엔드 UI 컴포넌트 개선
- **오늘 할 일:**
 - 가구 드래그앤드롭 기능 구현
 - 3D 뷰어 성능 최적화
 - 평면도와 3D 뷰어 동기화 작업
- **이슈:**
- **블로커:**

2.3.2.5. 7월 25일 (금)

- **어제 한 일:**
 - 가구 드래그앤드롭 기능 구현 완료

- 3D 뷰어 성능 최적화 (메모리 관리 개선)
- 평면도와 3D 뷰어 동기화 기본 구현
- **오늘 할 일:**
 - 전체 시스템 통합 테스트
 - 사용자 인터페이스 최종 개선
 - 3주차 스프린트 회고 및 문서 정리
- **이슈:**
- **블로커:**

2.4. 스프린트 4주차 (2025.07.28 ~ 08.01)

2.4.1. 4주차 EPIC : 팀 통합 작업 및 웹애플리케이션 완성

팀원들이 각자 개발한 컴포넌트와 기능들을 통합하여 완전한 웹애플리케이션을 완성한다. 전체 시스템의 일관성과 사용자 경험을 확보하고, 다음 주 배포를 위한 준비를 완료한다.

2.4.2. 일별 스크럼

2.4.2.1. 7월 28일 (월)

- **어제 한 일:** 3주차 스프린트 회고 및 문서 정리 완료
- **오늘 할 일:**
 - 팀원들과 웹페이지 통합 작업 시작
 - 프론트엔드 컴포넌트 통합 및 라우팅 구성
 - 백엔드 API 통합 및 데이터 플로우 연결
 - UI/UX 일관성 검토 및 조정
 - 발견된 버그 수정 및 성능 개선
 - 창문 감지 정확도 조정 필요
 - AI 벽 바닥 감지 조정 필요
- **이슈:**
- **블로커:**

2.4.2.2. 7월 29일 (화)

- **어제 한 일:** 팀원들과 웹페이지 통합 작업 시작, 프론트엔드 컴포넌트 통합 진행
- **오늘 할 일:**
 - 개발 컴포넌트 코드 리뷰
 - 전체 애플리케이션 네비게이션 구조 완성
 - 백엔드 분리
 - 공통 스타일 가이드 적용 및 CSS 통합
- **이슈:**

- 블로커:

2.4.2.3. 7월 30일 (수)

- 어제 한 일: 네비게이션 구조 완성, 백엔드 분리
- 오늘 할 일:
 - 백엔드 API 엔드포인트 통합 및 테스트
 - 프론트엔드-백엔드 데이터 통신 검증
 - 전체 사용자 플로우 테스트 (E2E 테스트)
 - 성능 최적화 및 로딩 속도 개선
 - 에러 핸들링 및 예외 처리 강화
- 이슈:
- 블로커:

2.4.2.4. 7월 31일 (목)

- 어제 한 일: 백엔드 API 통합 완료, 프론트엔드-백엔드 데이터 통신 검증, E2E 테스트
- 오늘 할 일:
 - 통합된 웹애플리케이션 최종 테스트
 - 사용자 인터페이스 폴리싱 및 디자인 완성
 - 반응형 웹 디자인 적용 및 테스트
 - 접근성(Accessibility) 개선 작업
 - 배포 준비를 위한 빌드 최적화
 - 시연 준비 및 발표 자료 작성
- 이슈:
- 블로커:

2.4.2.5. 8월 1일 (금)

- 어제 한 일: 통합 웹애플리케이션 최종 테스트, UI 폴리싱, 반응형 디자인 적용 완료
- 오늘 할 일:
 - 팀 전체 통합 결과 데모 및 검토
 - 최종 버그 픽스 및 코드 정리
 - 프로젝트 문서 업데이트 (README, API 문서 등)
 - Git 브랜치 정리 및 메인 브랜치 통합
 - 4주차 스프린트 회고 및 결과 정리
 - 5주차 계획 수립 (배포 환경 구축 중심)
- 이슈:
- 블로커:

2.5. 스프린트 5주차 (2025.08.04 ~ 08.08)

2.5.1. 5주차 EPIC : Docker 기반 배포 환경 구축 및 CI/CD 구성

Docker 컨테이너화와 CI/CD 파이프라인을 우선적으로 구축하여 안정적인 배포 기반을 마련한다. 컨테이너 환경에서의 애플리케이션 실행, 자동화된 빌드/배포 프로세스, 기본적인 모니터링 체계를 완성한다.

2.5.2. 일별 스크립

2.5.2.1. 8월 4일 (월)

- **어제 한 일:** 4주차 회고 및 5주차 계획 수립 (Docker 기반 배포 환경 구축 중심)
- **오늘 할 일:**
 - Docker 개발 환경 구축 및 Dockerfile 최적화
 - 프론트엔드/백엔드 컨테이너 분리 및 네트워킹
 - Docker Compose 다중 서비스 구성
 - 컨테이너 환경에서의 환경변수 관리
 - 로컬 개발 환경과 컨테이너 환경 동기화
- **이슈:**
- **블로커:**

2.5.2.2. 8월 5일 (화)

- **어제 한 일:** Docker 환경 구축, 컨테이너 분리 및 네트워킹, Docker Compose 구성 완료
- **오늘 할 일:**
 - GitHub Actions CI/CD 파이프라인 구축
 - Docker 이미지 자동 빌드 및 레지스트리 푸시
 - 환경별 배포 스크립트 작성 (dev, staging, prod)
 - 컨테이너 헬스체크 및 로그 수집 구성
 - 기본 모니터링 도구 연동 (Docker stats, logging)
- **이슈:**
- **블로커:**

2.5.2.3. 8월 6일 (수)

- **어제 한 일:** GitHub Actions CI/CD 파이프라인 구축, Docker 이미지 자동 빌드, 환경별 배포 스크립트 작성
- **오늘 할 일:**
 - 클라우드 환경 배포 테스트 (AWS/GCP/Azure)
 - 도메인 연결 및 SSL 인증서 설정
 - 환경별 데이터베이스 분리 및 연결

- 컨테이너 성능 최적화 및 리소스 관리
- 백업 및 복구 기본 전략 수립
- 이슈:
- 블로커:

2.5.2.4. 8월 7일 (목)

- **어제 한 일:** 클라우드 환경 배포 테스트, 도메인 및 SSL 설정, 환경별 데이터베이스 분리 완료
- **오늘 할 일:**
 - 프로덕션 환경 보안 설정 강화
 - 애플리케이션 로그 관리 및 분석 도구 연동
 - 기본 모니터링 대시보드 구성
 - 배포 자동화 스크립트 테스트 및 검증
 - 롤백 전략 및 장애 대응 매뉴얼 작성
- 이슈:
- 블로커:

2.5.2.5. 8월 8일 (금)

- **어제 한 일:** 프로덕션 보안 설정, 로그 관리 도구 연동, 모니터링 대시보드 구성, 배포 자동화 검증
- **오늘 할 일:**
 - 전체 Docker 기반 배포 파이프라인 종합 테스트
 - 배포 문서화 및 운영 가이드 작성
 - 팀원 배포 환경 교육 및 실습
 - 성능 테스트 및 최적화
 - 5주차 회고 및 6주차 계획 수립
- 이슈:
- 블로커:

2.6. 스프린트 6주차 (2025.08.11 ~ 08.15)

2.6.1. 6주차 EPIC : Kubernetes 도입 및 고급 배포 환경 구축

Docker 기반 환경에서 Kubernetes로 전환하여 확장성과 안정성을 높인다. 컨테이너 오케스트레이션, 오토스케일링, 서비스 메시, 고급 모니터링 등을 통해 엔터프라이즈급 배포 환경을 완성한다.

2.6.2. 일별 스크럼

2.6.2.1. 8월 11일 (월)

- **어제 한 일:** 5주차 회고 및 6주차 계획 수립 (Kubernetes 도입 중심)
- **오늘 할 일:**

- Kubernetes 클러스터 환경 구축 (minikube/EKS/GKE)
- 기존 Docker Compose를 K8s 매니페스트로 전환
- Deployment, Service, ConfigMap 기본 설정
- 네임스페이스 분리 및 리소스 관리
- 이슈:
- 블로커:

2.6.2.2. 8월 12일 (화)

- **어제 한 일:** AI 모델 앙상블 연구 및 다중 이미지 처리 시스템 설계
- **오늘 할 일:**
 - 실시간 스트리밍 분석 기능 개발
 - 캐싱 시스템 구축으로 성능 향상
 - 로드 밸런싱 및 오토스케일링 구현
- 이슈:
- 블로커:

2.6.2.3. 8월 13일 (수)

- **어제 한 일:** 실시간 스트리밍 분석 기능 개발 및 캐싱 시스템 구축
- **오늘 할 일:**
 - GPU 가속 최적화 작업
 - 분산 처리 시스템 구현
 - 모니터링 및 알림 시스템 고도화
- 이슈:
- 블로커:

2.6.2.4. 8월 14일 (목)

- **어제 한 일:** GPU 최적화 및 분산 처리 시스템 구현
- **오늘 할 일:**
 - A/B 테스트 시스템 구축
 - 데이터 파이프라인 최적화
 - 백업 및 복구 시스템 구현
- 이슈:
- 블로커:

2.6.2.5. 8월 15일 (금)

- **어제 한 일:** A/B 테스트 시스템 구축 및 데이터 파이프라인 최적화
- **오늘 할 일:**

- 6주차 종합 성능 테스트
- 시스템 안정성 검증
- 6주차 회고 및 최종 주차 계획 수립
- 이슈:
- 블로커:

2.7. 스프린트 7주차 (2025.08.18 ~ 08.22)

2.7.1. 7주차 EPIC : 최종 발표 준비 및 프로젝트 완성

8월 22일 최종 발표를 위한 종합적인 준비 작업을 수행한다. 프로젝트의 성과와 기술적 성취를 효과적으로 전달할 수 있는 발표 자료 제작, 시연 준비, 그리고 프로젝트 완성도를 최고 수준으로 끌어올린다.

2.7.2. 일별 스크럼

2.7.2.1. 8월 18일 (월)

- **어제 한 일:** 6주차 회고 및 최종 주차 계획 수립
- **오늘 할 일:**
 - 발표 준비 시작 - 프로젝트 전체 성과 정리
 - 발표 구조 및 목차 설계
 - 핵심 기술 및 혁신 포인트 정리
 - 시연 계획 수립 및 데모 환경 준비
 - 전체 시스템 종합 테스트 수행
- 이슈:
- 블로커:

2.7.2.2. 8월 19일 (화)

- **어제 한 일:** 종합 테스트 및 문서 작업 진행
- **오늘 할 일:**
 - 발표 스토리보드 및 시나리오 구체화
 - 프로젝트 하이라이트 영상 제작
 - 기술적 도전과제 및 해결과정 정리
 - 향후 발전 방향 및 로드맵 작성
 - 발표 연습 및 팀원 피드백 수집
- 이슈:
- 블로커:

2.7.2.3. 8월 20일 (수)

- **어제 한 일:** 최종 사용자 테스트 및 런칭 준비 작업
- **오늘 할 일:**
 - 발표용 시연 데모 시나리오 작성
 - 프로젝트 성과 지표 및 통계 정리
 - 기술 스택 및 아키텍처 다이어그램 준비
 - 발표 PPT 초안 작성
 - 팀 역할 및 기여도 정리
- **이슈:**
- **블로커:**

2.7.2.4. 8월 21일 (목)

- **어제 한 일:** 프로덕션 환경 점검 및 마케팅 자료 준비
- **오늘 할 일:**
 - 최종 발표 자료 완성 및 검토
 - 발표 시연 시나리오 최종 점검
 - 팀 발표 역할 분담 및 연습
 - 기술적 질문 대비 답변 준비
 - 데모 환경 안정성 최종 확인
- **이슈:**
- **블로커:**

2.7.2.5. 8월 22일 (금) - 최종 발표일

- **어제 한 일:** 소프트 런칭 및 초기 모니터링
- **오늘 할 일:**
 - 🎯 최종 발표 및 시연 (D-Day)
 - 발표 자료 최종 점검 및 리허설
 - 실시간 데모 환경 최종 확인
 - Q&A 세션 대비 답변 준비
 - 프로젝트 성과 및 기술적 성취 정리
 - 팀원별 기여도 및 학습 성과 발표
- **이슈:**
- **블로커:**